

Term Project

- ▶ All projects need to be approved before starting
- ▶ Teams will need to submit a Term Project Proposal DRAFT
 - ▶ Due next week (2/11)
 - ▶ [Assignments > Term Project > Term Project Proposal – Draft](#)
 - ▶ Due 2/18
 - ▶ [Assignments > Term Project > Term Project Proposal – Final](#)
- ▶ Reach out if you have any questions
- ▶ All teams will do a project proposal presentation
 - ▶ 5-minute presentation outlining their project idea to the class

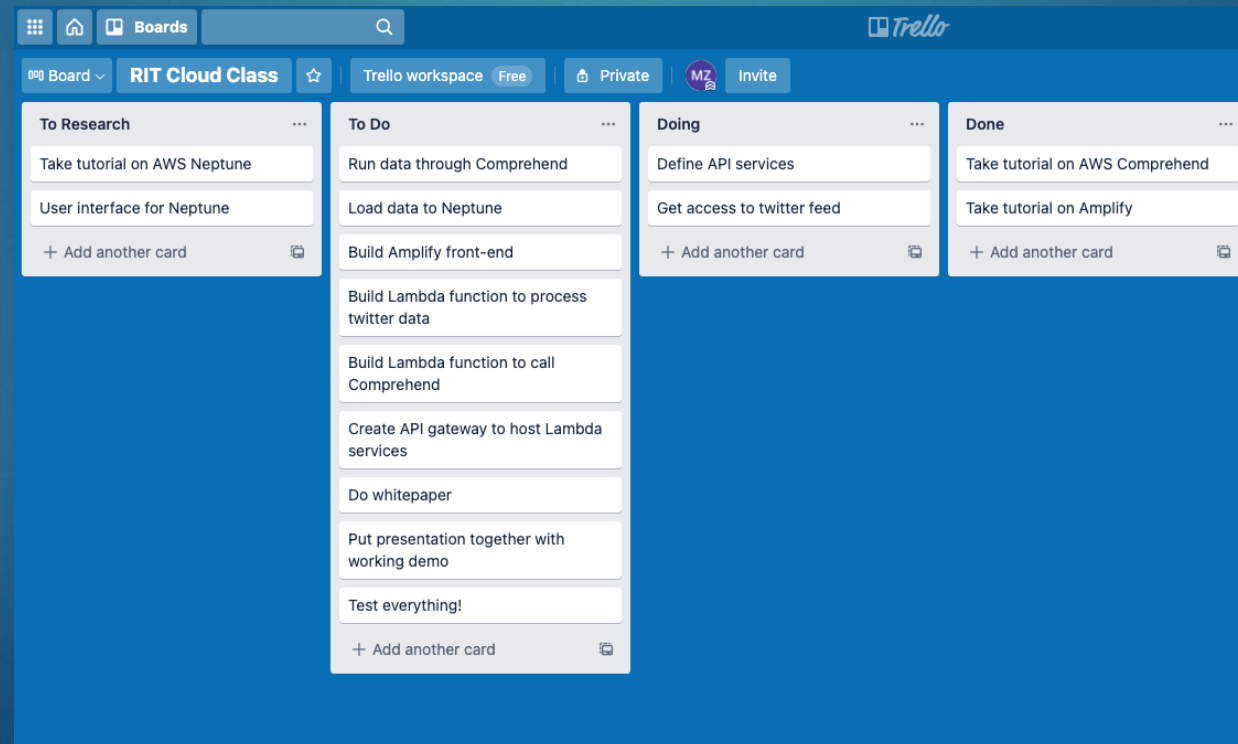
CATEGORY	EXCEEDS EXPECTATIONS (76-100%)	MEETS EXPECTATIONS (51-75%)	PARTIALLY MEETS EXPECTATIONS (26-50%)	DID NOT MEET EXPECTATIONS (0-25%)	TOTAL
TEAM PROJECT PROPOSAL PRESENTATION	Proposal clearly and succinctly articulates the project's intent, goals, and the AWS technologies to be used.	Proposal states the project's intent and goals but includes minor issues with clarity or AWS technology integration.	Proposal outlines the project but is unclear about the goals or lacks details about the AWS technologies.	Proposal is vague, missing key details about the project's goals and AWS technologies.	5
TEAM CHECK-INS	See Project Check-In document in myCourses, which details deliverables for each of the 3 check-ins				20
USER'S GUIDE	User's guide is clear, complete, and includes all necessary instructions for project setup and use.	User's guide is clear but may have minor ambiguities around setup or usage.	User's guide is incomplete or unclear, causing confusion about setup or usage.	User's guide is absent or fails to provide actionable instructions.	10
FINAL PRESENTATION	Presentation clearly defines the problem and solution, includes cost analysis, and showcases a polished, fully functional demo.	Presentation defines the problem and solution well but includes minor clarity issues or a demo with small technical hiccups.	Presentation lacks clarity, misses key details about the problem or solution, or the demo encounters significant technical problems.	Presentation is unclear or incomplete, and the demo is either missing or fails to function effectively.	20
PROJECT SETUP & TEARDOWN	Project setup and teardown are automated using GitHub Actions, executing seamlessly with no issues. Documentation is clear and complete, enabling automated workflows without assistance.	Project setup and teardown using GitHub Actions run successfully with minor, easily resolved issues. Documentation is mostly clear with a few needed clarifications.	Significant issues occurred during project setup or teardown using GitHub Actions, requiring team assistance. Documentation was incomplete or unclear, leading to confusion.	Project setup or teardown failed, or GitHub Actions was not utilized as required. Documentation was missing or lacked clarity, requiring substantial team intervention.	20
OVERALL EFFORT, DESIGN & EXECUTION	The team used all 8 required AWS technologies and integrated at least one additional AWS service in a meaningful way. The project demonstrated strong technical depth, thoughtful design, and went beyond the assignment's core requirements. Execution was complete, polished, and showed clear evidence of extra effort and innovation.	The team used all 8 required AWS technologies and may have included an additional service. The solution met assignment requirements with acceptable complexity and effort, though it remained within standard expectations. Execution was mostly complete, with only minor gaps or missed opportunities for enhancement.	The team used all 8 required AWS technologies, but the solution lacked expected complexity, depth, or integration. Implementation showed limited effort, minimal technical design, or noticeable gaps in functionality. Project goals were only partially met.	The team used 8 or fewer required AWS technologies, but the solution was either incomplete, poorly integrated, or lacking in complexity. Significant execution issues were present, and the project did not meet core objectives or reflect sufficient effort.	25
BONUS POINTS	See Term Project document regarding details to earn additional bonus points				10
TOTAL POINTS					110

Team Check-ins

- ▶ There will be 3 check-ins for the term project, totaling 20 points
 - ▶ Check-in 1 & 2 = 5 points each
 - ▶ Check-in 3 = 10 points
 - ▶ See Topic Schedule (on myCourses) for the dates of each check-in
- ▶ There will be deliverables due for each check-in
- ▶ All teams should schedule a 10 - 15 minute appointment with the instructor or CA to review team progress for the term project
- ▶ All team members are encouraged to attend though it is acceptable to designate a representative(s) to speak on behalf of the team
- ▶ This is your opportunity to clarify any issues/questions and expectations for the project

Team Check-ins

- ▶ All teams must track their progress using Trello using a simple Kanban style
- ▶ I will send out a board template to each team
- ▶ Teams are expected to keep these up to date and be prepared to show their board for each check-in



- ▶ How long did you take to complete the WordPress or the VPC and Subnets activity?
- ▶ Along the way, it can be easy to make a typing mistake
- ▶ This process isn't automated so doing this all over again can be highly error prone and your results could be different
- ▶ There must be an easier way?



Infrastructure as Code / Introduction to Terraform

Engineering Cloud Software Systems

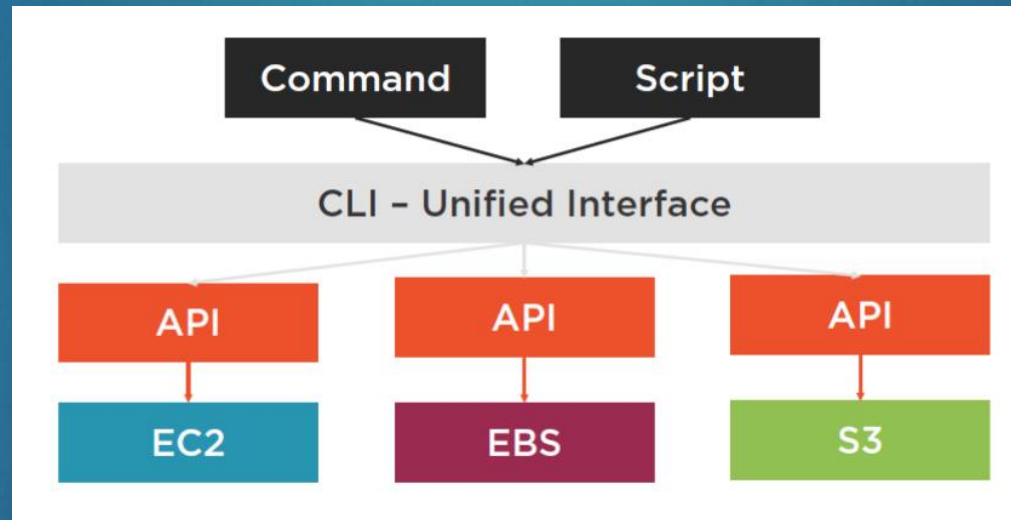
Department of Software Engineering
Rochester Institute of Technology



AWS CLI – Command-line Interface

7

- ▶ The AWS Command Line Interface (CLI) is a unified tool to manage your AWS services
- ▶ It provides functionality equivalent to the AWS Management Console



- ▶ Download required for Windows, Mac & Linux, but runs natively on Amazon Linux AMI, CloudShell and Learner Labs CLI

AWS CLI – Examples

- ▶ If you have installed AWS CLI on your PC, you will need to do this
 - ▶ Initial set up, requires access to your secret key

```
$ aws configure
AWS Access Key ID [None]: AKI[REDACTED]PLE
AWS Secret Access Key [None]: je7MtGbClwBF/2Zp[REDACTED]EXAMPLEKEY
Default region name [None]: us-east-1
Default output format [None]: json
```

- ▶ You can now access your AWS resources
- ▶ What are my EC2 instance?

```
$ aws ec2 describe-instances
```

- ▶ Start an EC2 instance

```
$ aws ec2 start-instances --instance-ids i-1348636c
```


AWS CLI – Examples

► Create a new EC2 instance

```
$ aws ec2 run-instances --image-id ami-01c647eace872fc02 --count 1 --instance-type t2.micro --key-name [YOUR KEY NAME]
```

Use your key name from key pair name on AWS, not from your computer

Amazon Machine Image (AMI)

Amazon Linux 2 AMI (HVM) - Kernel 5.10, SSD Volume Type

ami-05fa00d4c63e32376 (64-bit (x86)) / ami-05f3141013eebdc12 (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible ▼

- There are many more arguments you can specify for your EC2 instances
- For more information, refer to the help

```
$ aws ec2 run-instances help
```

AWS CLI – Examples

10

- ▶ Create a new S3 bucket (mb = make bucket)

```
$ aws s3 mb s3://bucket-name
```

- ▶ List all buckets

```
$ aws s3 ls  
2018-12-11 17:08:50 my-bucket  
2018-12-14 14:55:44 my-bucket2
```

- ▶ Delete a bucket (rb = remove bucket)

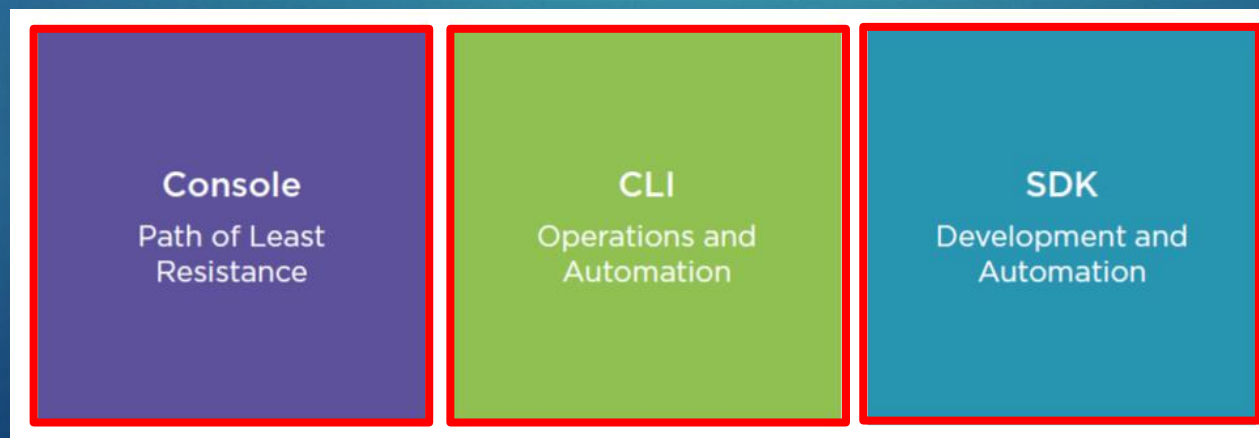
```
$ aws s3 rb s3://bucket-name
```

- ▶ Copy a file from my PC to my bucket

```
$ aws s3 cp myvideo.mp4 s3://bucket-name/
```

AWS CLI - Summary

- ▶ AWS CLI gives you the ability to automate the entire process of controlling and managing AWS services through scripts
- ▶ These scripts make it easy for users to fully automate cloud infrastructure
- ▶ There are many more commands available, which provide a very granular level of control over your resources
- ▶ If you prefer using a programming language (Java, Python, .NET, Ruby, etc.), take a look at the [AWS SDKs](#)



SDK Example - Python

- ▶ AWS boto3 is the Python SDK for AWS

```
$ python

Python 3.9.4 (v3.9.4:1f2e3088f3, Apr  4 2021, 12:32:44)
[Clang 6.0 (clang-600.0.57)] on darwin
Type "help", "copyright", "credits" or "license" for more information.

>>> import boto3
>>> ec2_client = boto3.client("ec2", region_name="us-east-1")
>>> instances = ec2_client.run_instances(
...     ImageId="ami-0e9089763828757e1",
...     MinCount=1,
...     MaxCount=1,
...     InstanceType="t2.micro",
...     KeyName="[YOUR KEY NAME]"
... )
>>>
```



Boto 3

Beyond CLI & SDK

- ▶ For smaller setups on AWS, CLI and SDK can be a good solution
 - ▶ Using a series of scripts and/or SDKs, you can quickly provision infrastructure
 - ▶ It's easy to understand
 - ▶ Scripts can be managed in an SCM like Git
- ▶ For larger, more complex setups on AWS that require provisioning multiple AWS resources beyond EC2 and S3 (load balancers, databases, message queues, etc), this can be increasingly more difficult to manage
 - ▶ Setup of some services may need to be done in a specific order
 - ▶ A few more challenges will also become more apparent...



Challenges with More Complex Systems

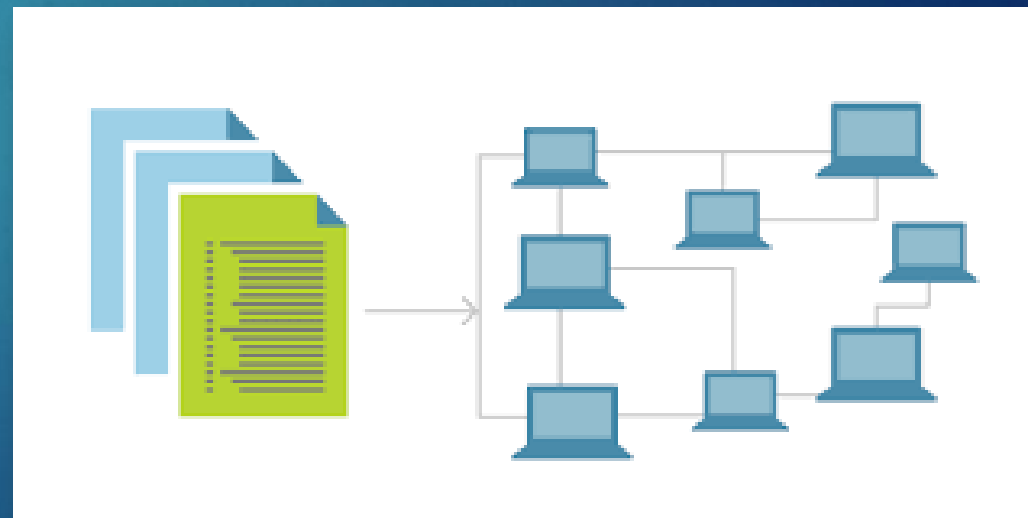
14

- ▶ I can't manually stop every resource each day, so I need a reliable way to tear everything down and recreate it the next day
- ▶ I want resources to start up in parallel wherever possible to reduce wait times
- ▶ If a startup process fails, I need clear visibility into the cause and an automated rollback of any provisioned resources to avoid unnecessary costs
- ▶ Manual provisioning is error-prone, so I want to eliminate the need for repetitive manual steps
- ▶ I need a repeatable, automated process for provisioning and managing resources



Infrastructure as Code (IaC)

- ▶ Automates infrastructure management using software development practices
- ▶ Treats servers, databases, and networks as code that can be versioned and tested
- ▶ Enables consistent, repeatable provisioning and configuration
- ▶ Applies changes through automated workflows with validation
- ▶ Improves speed, reliability, scalability, and reduces human error



IaC - Benefits

- ▶ Speed & Simplicity
 - ▶ Quickly provision complete infrastructure (servers, databases, storage, etc.) by running a script
- ▶ Configuration Consistency
 - ▶ Standardizes infrastructure setup, reducing errors and ensuring predictable environments
- ▶ Cost Efficiency
 - ▶ Automates spinning down unused environments, reducing unnecessary cloud costs
- ▶ Elimination of Configuration Drift
 - ▶ IaC is idempotent—every deployment produces the same result, ensuring stable, reproducible environments
- ▶ Unified Deployment
 - ▶ Application code and infrastructure can be deployed together in a single automated process



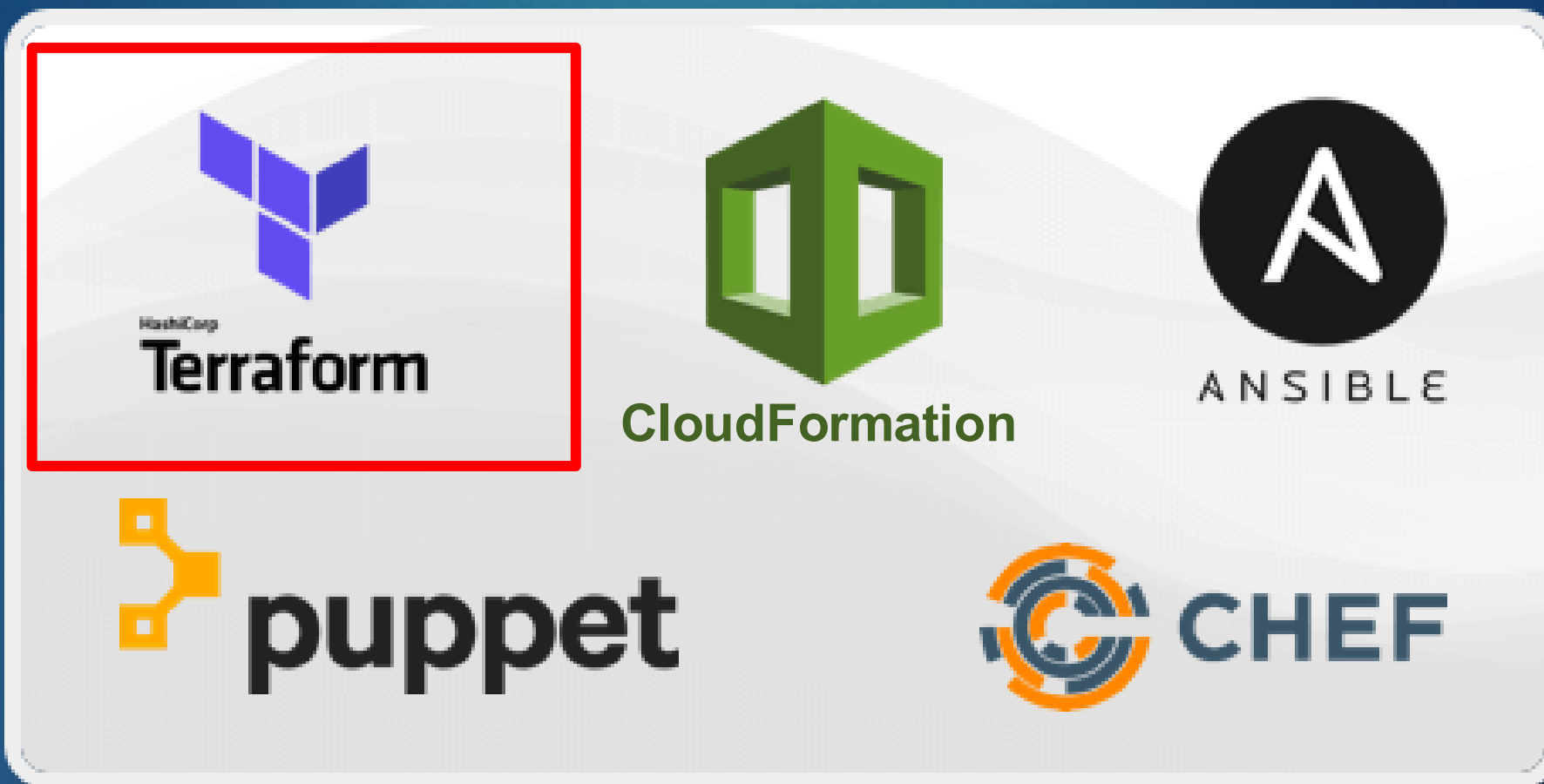
IaC – Remember...It's Just Code

- ▶ Defining infrastructure as code enables you to manage it the same way as application code
- ▶ Infrastructure definitions can be authored in any code editor and stored in version control for tracking and teamwork
- ▶ Just as the same source code produces the same binary, an IaC model consistently generates the same environment each time it's applied



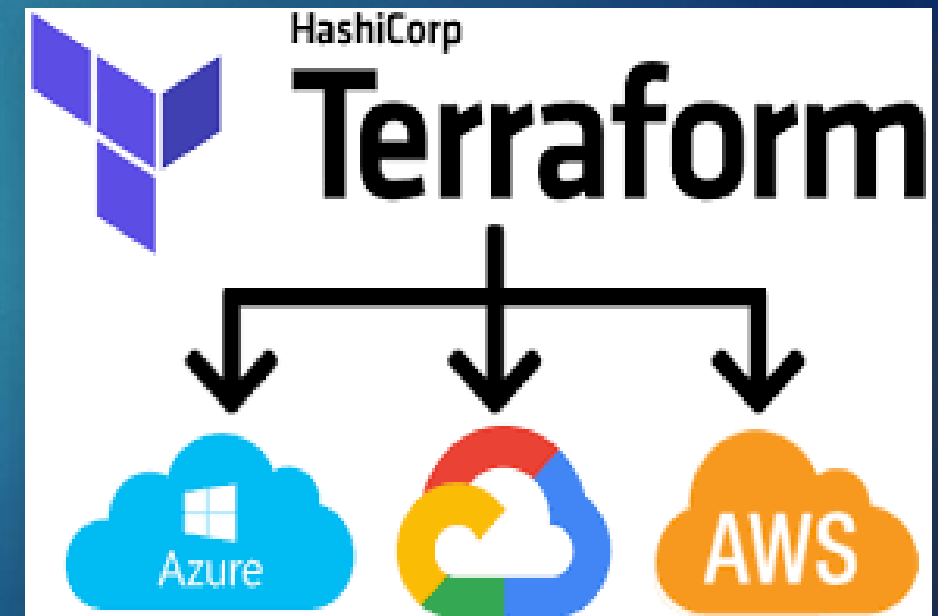
IaC - Tooling

18



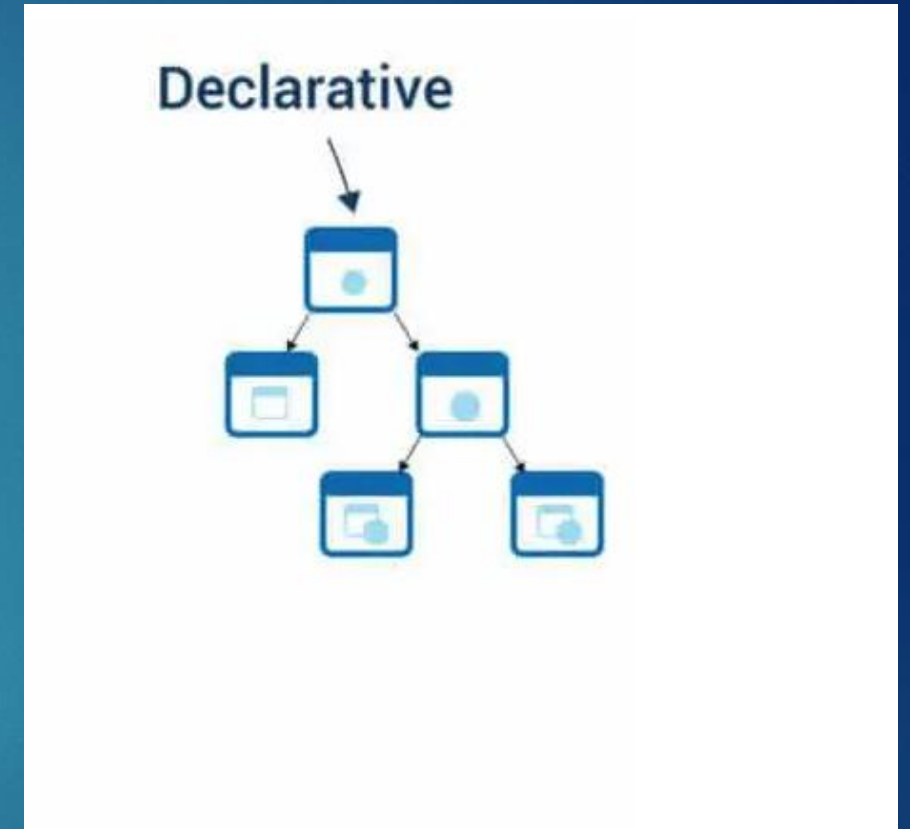
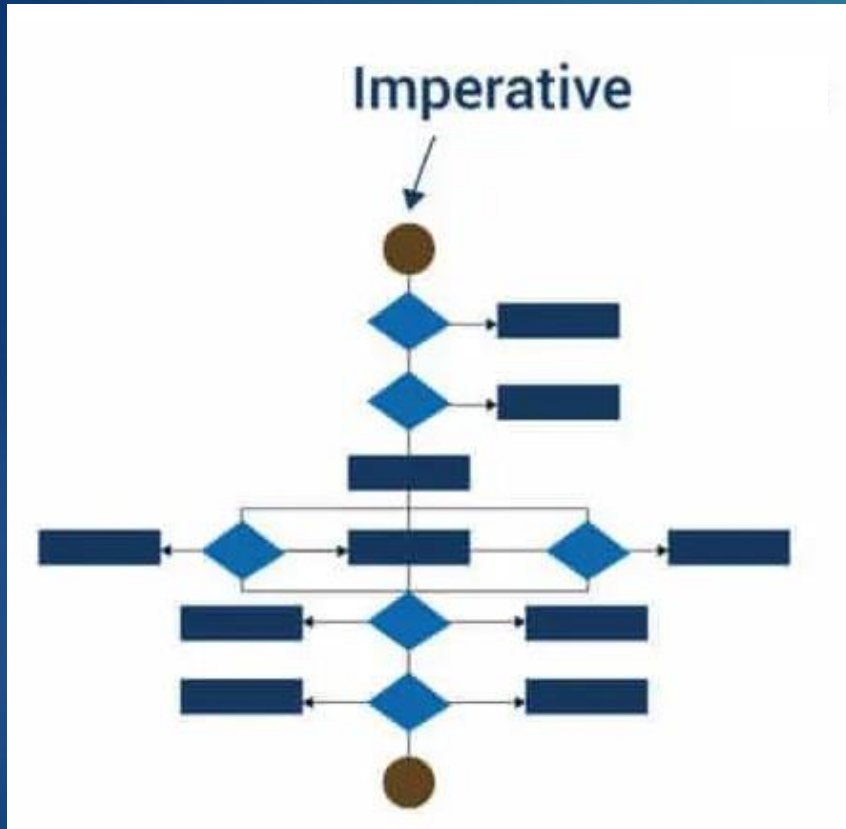
What is Terraform?

- ▶ Terraform is an Infrastructure as Code (IaC) tool created by HashiCorp
- ▶ Terraform can manage resources across a wide range of providers, including AWS, Azure, GCP, and many more
- ▶ You do not need to write step-by-step instructions to create resources because Terraform handles the underlying logic
- ▶ It uses a declarative syntax



Declarative vs. Imperative

20



- ▶ Tell the system **HOW** to do something – every step of the way (setup and teardown)
- ▶ Examples: AWS CLI, SDK

- ▶ Tell the system **WHAT** you want and let it figure out **HOW** to do it
- ▶ Examples: CloudFormation, CDK, Terraform

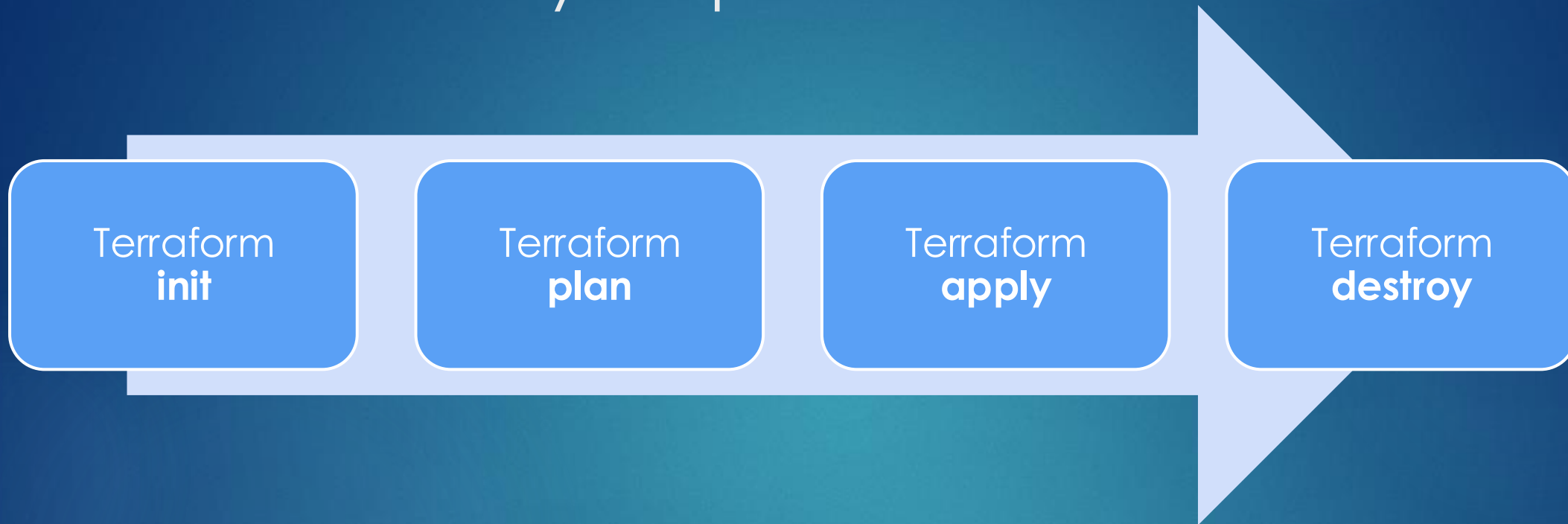
Key Features



- ▶ Provider Agnostic
 - ▶ Terraform supports a wide range of providers (e.g., AWS, Azure, Google Cloud, etc.), allowing users to manage infrastructure across multiple platforms with a consistent approach
- ▶ Resource Graph
 - ▶ Terraform builds a dependency graph of all resources, ensuring they are created or modified in the correct order
- ▶ State Management
 - ▶ Terraform maintains the state of the infrastructure, allowing it to track changes and apply only the necessary updates
- ▶ Modularity
 - ▶ Configurations can be organized into modules, enabling code reuse and easier management of complex infrastructures
- ▶ Community and Ecosystem
 - ▶ Terraform has a strong community and a rich ecosystem of modules, plugins, and integrations that enhance its functionality and ease of use

Terraform Key Steps

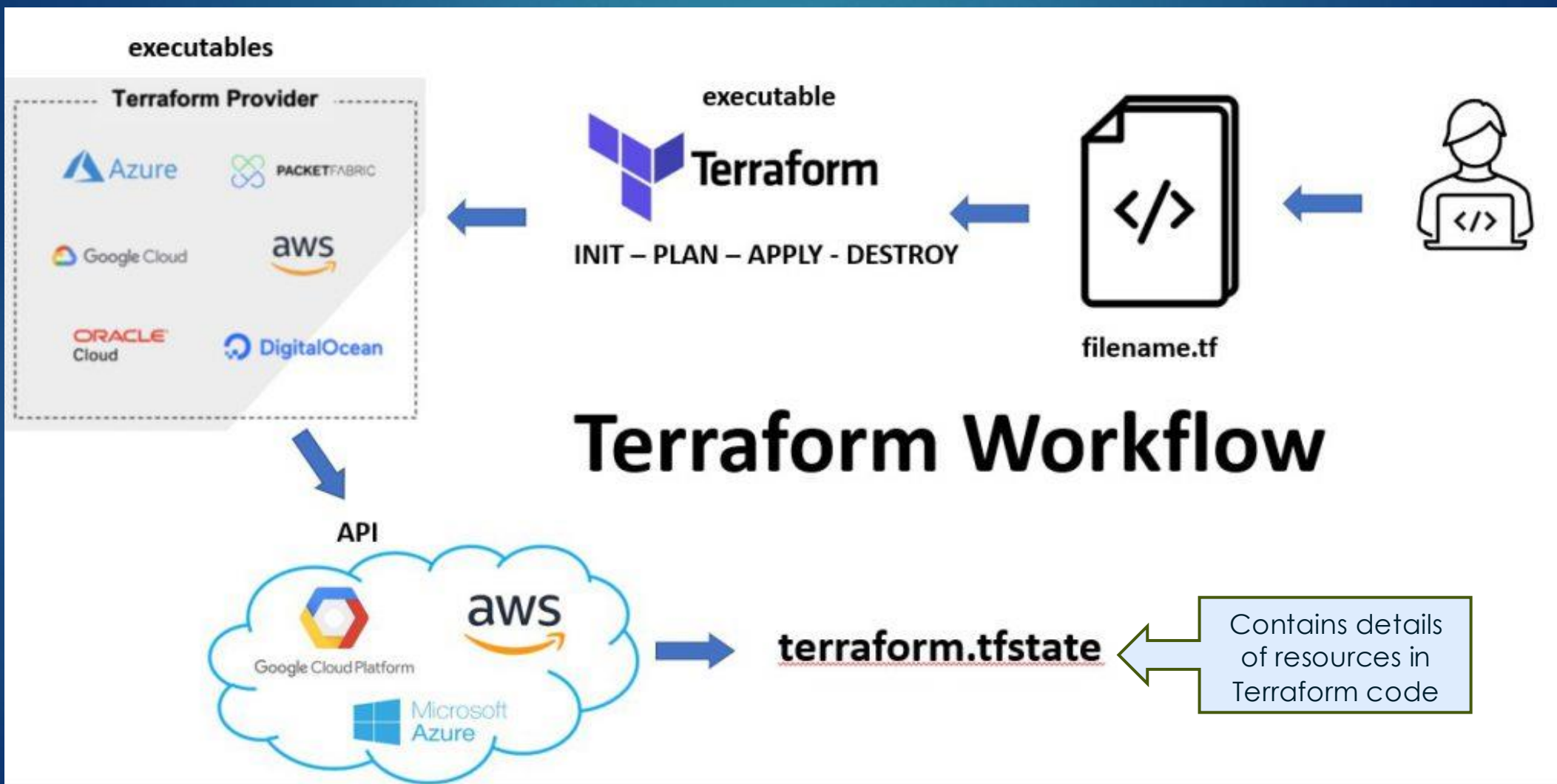
22



- ▶ **init** – Initialize the (local) Terraform environment. Usually executed only once per session.
- ▶ **plan** – Compare the Terraform state with the as-is state in the cloud, build and display an execution plan. This does not change the deployment (read-only).
- ▶ **apply** – Apply the plan from the plan phase. This potentially changes the deployment.
- ▶ **destroy** – Destroy all resources that are governed by this specific terraform environment.

Terraform Workflow

23



Terraform Core Concepts

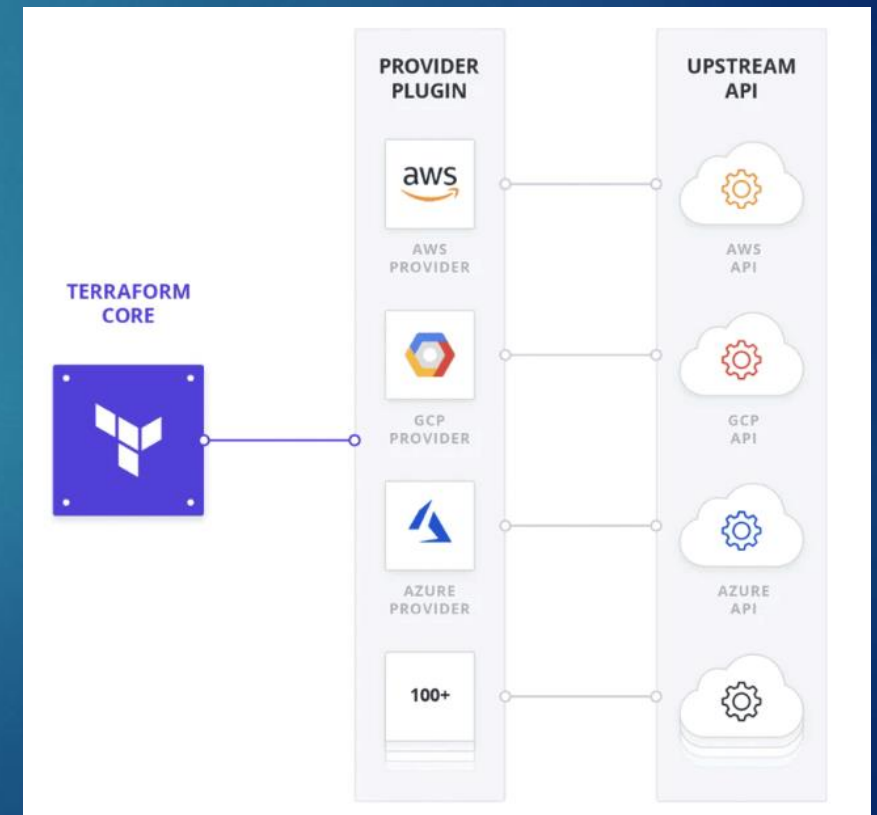
- ▶ Providers
- ▶ Resources
- ▶ Modules
- ▶ Data sources
- ▶ Input Variables
- ▶ Output Values



Terraform Core Concepts - Providers

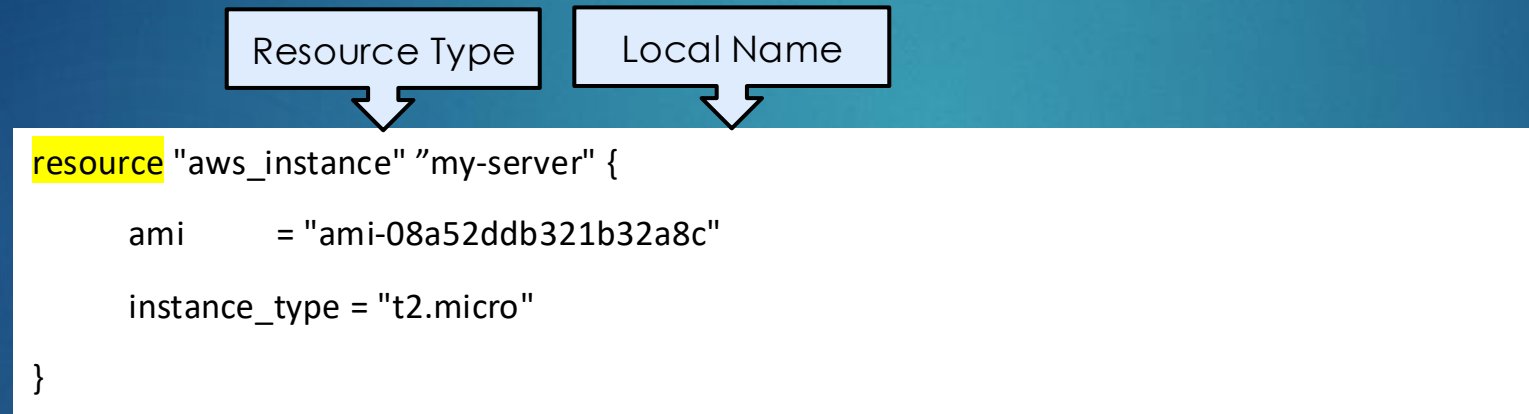
- ▶ A provider is a plugin that interacts with the various APIs required to create, update, and delete various resources
- ▶ Each provider adds a set of resource types and/or data sources that Terraform can manage

```
provider "aws" {  
  access_key = "YOUR_AWS_ACCESS_KEY"  
  secret_key = "YOUR_AWS_SECRET_KEY"  
  region    = "us-east-1"  
}
```



Terraform Core Concepts - Resources

- ▶ Resources are the most important element in the Terraform language
- ▶ Each resource block describes one or more infrastructure objects, such as storage, virtual networks or compute instances



- ▶ Each resource type has its own attributes to define that specific resource
- ▶ The local name is used to refer to this resource from elsewhere in the same Terraform module, but has no significance outside that module's scope

Terraform Core Concepts – Input Variables 27

- ▶ Variables are used as parameters to input values at run time to customize our deployments

```
variable "ami" {  
  type    = string  
  description = "AMI ID for the EC2 instance"  
  default  = "ami-08a52ddb321b32a8c"  
}  
  
variable "type" {  
  type    = string  
  description = "Instance type for the EC2 instance"  
  default  = "t2.micro"  
}
```

```
resource "aws_instance" "my-server" {  
  ami      = var.ami  
  instance_type = var.type  
}
```

- ▶ Within the resource blocks, we have simply used these variables by using `var.<variable name>` format
- ▶ When you proceed to plan and apply this configuration, the variable values will automatically be replaced by **default** values

Terraform Core Concepts – Data Source

28

- ▶ In Terraform, a data source represents data that is fetched from outside of Terraform itself, or data that is computed within Terraform but not managed by it
- ▶ It allows Terraform to read data from a given provider, and use that data elsewhere in the Terraform configuration
- ▶ It helps to eliminate hard-coding of information

Terraform will perform the following actions:

```
# aws_instance.my_server will be created
+ resource "aws_instance" "my_server" {
+   ami = "ami-08a52ddb321b32a8c"
```

Amazon Linux 2023 AMI
ami-08a52ddb321b32a8c (64-bit (x86)) / ami-0decaa87ef4609834 (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

```
data "aws_ami" "amazonlinux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name = "name"
    values = ["al2023-ami-2023*"]
  }

  filter {
    name = "virtualization-type"
    values = ["hvm"]
  }

  filter {
    name = "root-device-type"
    values = ["ebs"]
  }

  filter {
    name = "architecture"
    values = ["x86_64"]
  }
}

resource "aws_instance" "my-server" {
  ami = data.aws_ami.amazonlinux.id
  instance_type = var.type
}
```

Identifies latest AMI available

A **filter** must return a single value or it will fail

Terraform Core Concepts – Locals

- ▶ Terraform local values (aka locals) enable the creation of expressions or values that can be easily referenced within the Terraform module
- ▶ Using local values helps eliminate the duplication of hard coded values that are used multiple times throughout the Terraform project

```
resource "aws_instance" "my-server" {  
  ami = data.aws_ami.amazonlinux.id  
  instance_type = var.type  
  key_name = "MZ-AWS-KEYS"  
}
```

NOTE: A local is only accessible within the module vs a Terraform variable which can be scoped globally

```
locals {  
  aws_key = "MZ-AWS-KEY"  
}  
  
resource "aws_instance" "my-server" {  
  ami = data.aws_ami.amazonlinux.id  
  instance_type = var.type  
  key_name = "${local.aws_key}"  
}
```

- ▶ If you are creating multiple resources that require your key name, this will need to be repeated throughout your modules

- ▶ Create a local value for your key name so that it can be referenced everywhere

Terraform Core Concepts – Output Values

30

- ▶ Output values make information about your infrastructure available on the command line, and can expose information for other Terraform configurations to use
- ▶ Output values are similar to return values in programming languages

```
resource "aws_instance" "my_server" {  
  ami      = data.aws_ami.amazonlinux.id  
  instance_type = var.type  
}  
  
output "instance_ip_addr" {  
  value = aws_instance.my_server.private_ip  
}
```

For multiple servers, use:
aws_instance.my_server[*].private_ip

```
aws_instance.my_server: Creating...  
aws_instance.my_server: Still creating... [10s elapsed]  
aws_instance.my_server: Still creating... [20s elapsed]  
aws_instance.my_server: Still creating... [30s elapsed]  
aws_instance.my_server: Creation complete after 33s [id=i-0500c8f03afcc1767]  
  
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.  
  
Outputs:  
instance_ip_addr = "172.31.44.47"
```

Tying it all together

main.tf

```
provider "aws" {  
  region = "us-east-1"  
}  
  
locals {  
  aws_key = "MZ-AWS2024-KEY"  
}  
  
data "aws_ami" "amazonlinux" {  
  most_recent = true  
  owners     = ["amazon"]  
  
  filter {  
    name = "name"  
    values = ["al2023-ami-2023*"]  
  }  
  filter {  
    name = "virtualization-type"  
    values = ["hvm"]  
  }  
  filter {  
    name = "root-device-type"  
    values = ["ebs"]  
  }  
  filter {  
    name = "architecture"  
    values = ["x86_64"]  
  }  
}  
  
variable "type" {  
  type      = string  
  description = "Instance type for the EC2 instance"  
  default   = "t2.micro"  
}  
  
resource "aws_instance" "my_server" {  
  ami          = data.aws_ami.amazonlinux.id  
  instance_type = var.type  
  key_name     = "${local.aws_key}"  
}  
  
output "instance_ip_addr" {  
  value = aws_instance.my_server.private_ip  
}
```

- This demonstrates all the core concepts of Terraform to create an EC2 instance



WordPress revisited with Terraform

- ▶ Do the activity in [Assignments > Activity - Terraform and WordPress](#)
- ▶ Part 1 – Due by end of class
 - ▶ You will be provided with the code in this lecture to create an EC2 server
- ▶ Part 2 – Due next class
 - ▶ With the supplied code, setup WordPress from the [Install WordPress on EC2](#) activity
 - ▶ This must be fully automated through Terraform

